

Task-01: POS Order & Inventory System

Concurrency-Safe Order Processing | Techloom.ai Intern Assessment

1. Technical Specifications

Component	Technology	Notes
Backend	Node.js + Express.js	REST API, handles all business logic
Frontend	React.js (Vite)	SPA with Tailwind CSS for styling
Database	MySQL	Relational DB with transactions & row locking
Deployment - Frontend	Vercel	Auto-deploy from GitHub
Deployment - Backend	Railway	Node.js service with env variables
Deployment - Database	Railway MySQL Plugin	Attached to backend service
Version Control	GitHub	Public repository, single repo with /task-01 folder

2. Architecture Overview

Single backend architecture using Node.js Express. The frontend (React.js) communicates with the backend via REST API. The backend performs all database operations using MySQL with transactions to ensure data integrity and concurrency safety.

Concurrency Strategy: All stock deductions use MySQL transactions with SELECT ... FOR UPDATE row-level locking to prevent race conditions when multiple cashiers attempt to purchase the same limited-stock product simultaneously.

Reservation Timer: A background cron job (node-cron) runs every 60 seconds and scans the reservations table for entries where expires_at has passed. Expired reservations are released and stock is restored automatically.

3. Folder Structure

```
task-01/
├── Frontend/
│   ├── public/
│   └── src/
│       ├── pages/
│       │   ├── POSDashboard.jsx      # Main POS screen - inventory, item
│       │   ├── details, slip, timer
│       │   ├── Checkout.jsx          # Payment methods - cash and card
│       │   ├── PaymentSuccess.jsx    # Success screen with order summary +
│       │   ├── redirect timer
│       │   ├── PaymentFailed.jsx     # Failed screen with retry or cancel
│       │   └── options
│       │       ├── Cancel.jsx        # Order cancellation page
│       └── components/
│           └── InventoryTable.jsx    # Displays Items with Stock and Price
```

```
LKR columns
| | | | ItemDetails.jsx # Shows selected items, quantity, line
totals
| | | | SlipPreview.jsx # Receipt preview with barcode
| | | | ReservationTimer.jsx # Countdown timer - "Stocks Occupied
for X:XX"
| | | | CashPayment.jsx # Total / Paid amount / Balance amount
form
| | | | CardPayment.jsx # Card number, name, expiry, CVV, Scan
Card
| | | | Sidebar.jsx # Navigation sidebar
| | | | Footer.jsx # Footer component
| | | services/
| | | | productService.js # API calls for product/inventory CRUD
| | | | orderService.js # API calls for order creation,
cancellation
| | | | reservationService.js # API calls for stock reservation and
release
| | | | paymentService.js # API calls for mock payment processing
| | | styles/
| | | | globals.css
| | | App.jsx
| | | main.jsx
| | | index.css
| | index.html
| vite.config.js
| tailwind.config.js
| package.json
Backend/
| config/
| | db.js # MySQL connection pool config
| controllers/
| | productController.js # CRUD for products/inventory
| | orderController.js # Create, cancel, get orders
| | reservationController.js # Reserve stock, release, expiry cron
| | paymentController.js # Mock payment: success/fail/timeout
| routes/
| | productRoutes.js
| | orderRoutes.js
| | reservationRoutes.js
| | paymentRoutes.js
| middleware/
| | errorHandler.js # Global error handling middleware
| | validateRequest.js # Input validation middleware
| models/
| | Product.js # Product model/queries
| | Order.js # Order model/queries
| | Reservation.js # Reservation model/queries
| | Payment.js # Payment model/queries
| app.js # Express app setup, routes, middleware
| package.json
README.md # Setup, env vars, deployment links,
test guide
```

4. UI Pages & Mockup Flow

POS System

Inventory

Items	Stock	Price LKR
Apple	200	250
Noodles	65	120
Biscuit	34	100
Chocolate	279	150
Bread	23	150
Tap	56	50
Book	42	450
Monitor	4	11500
Pencil	689	40

Item Details

Items	Quantity	Price LKR
Apple	1	250
Noodles	1	120
Biscuit	1	100
Chocolate	2	150
Total		150

Stocks Occupied for
3:57

Proceed to Checkout

Slip Preview

AROMEX
Address

QTY	DESC	AMT
1	Item 1	000.00
1	Item 2	000.00
	TOTAL	000.00



Figure 1:POS Dashboard

POS System

Check out details

Items	Quantity	Price LKR
Apple	3	250
Noodles	1	120
Biscuit	2	100
Chocolate	2	150
Total		1370

Payment Methods

Pay with cash

Total1370.00

Paid amount1500.00

Balance amount130.00

Pay with card

Scan Card

Name on Card
John Newman

Card number
4755131301899638

Expiration date
09/27

CVV
3-4 digit code

Slip Preview

AROMEX
Address

QTY	DESC	AMT
1	Item 1	000.00
1	Item 2	000.00
	TOTAL	000.00



Figure 2:Checkout Page

4.1 POS Dashboard (POSDashboard.jsx)

Main screen used by cashiers. Divided into three panels:

LEFT PANEL - Inventory Section: Displays a table with columns: Items | Stock | Price LKR. Shows all products with current stock count and price. Cashier can see stock availability at a glance.

CENTRE PANEL - Item Details Section: Item name search bar with Search / Add / Remove buttons. Quantity input with Add / Remove buttons. Selected items appear in a table: Items | Quantity | Price LKR with a running Total. Red countdown timer at bottom: 'Stocks Occupied for X:XX' - starts when first item is added to the order.

RIGHT PANEL - Slip Preview: Live receipt preview showing store name, date, cashier, items with amounts, subtotal, tax, balance, and barcode. Updates as items are added.

Action Button: 'Proceed to Checkout' navigates to the Checkout page.

4.2 Checkout Page (Checkout.jsx)

Displayed after 'Proceed to Checkout'. Divided into three panels:

LEFT PANEL - Checkout Details: Final list of items with Quantity and Price LKR. Bold Total row at bottom.

CENTRE PANEL - Payment Methods:

Pay with Cash: Total field (auto-filled), Paid Amount input (cashier enters cash given), Balance Amount (auto-calculated = Paid - Total). Cancel and Done buttons.

Pay with Card: Scan Card button (simulates card scan). Manual card form: Name on Card, Card Number, Expiration Date, CVV, USE THIS CARD button. Cancel and Done buttons.

RIGHT PANEL - Slip Preview: Final receipt preview. On Done, slip is complete with payment confirmation.

4.3 Payment Success (PaymentSuccess.jsx)

Displayed after successful payment (Done clicked with valid payment).

Shows: 'Your Payment Success' in green text. Order Summary box with items list and total in LKR. Auto-redirect countdown 'You will be redirected to home in X:XX'. Back to Home button.

Backend action: Order status set to Paid. Reservation removed. Stock permanently deducted.

4.4 Payment Failed (PaymentFailed.jsx)

Displayed when mock payment gateway returns a failure response.

Shows: 'Your Payment Failed' in red text. Order Summary box with items and total. Message: 'Payment method failed. Stock has been released.' Two buttons: Update

Payment Method (returns to Checkout) | Cancel Order (cancels order and releases stock).

Backend action: Order status set to Failed. Reservation released immediately. Stock restored.

4.5 Order Cancellation (Cancel.jsx)

Can be reached from Payment Failed screen or cashier manually cancels an active order.

LEFT PANEL: Item image placeholder, Item details label, Reason input field, Select Reason dropdown button, Cancel Order button.

RIGHT PANEL: (For paid orders that need refund) Refund Methods section with Card and Bank Transfer options, each with a Select button.

Backend action: Order status set to Cancelled. Reserved or paid stock is restored to available inventory.

5. Database Design

Database name: pos_database

5.1 products table

```
CREATE TABLE products (  
  product_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  price DECIMAL(10, 2) NOT NULL,  
  stock_quantity INT NOT NULL DEFAULT 0,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP  
);
```

5.2 cashiers table

```
CREATE TABLE cashiers (  
  cashier_id INT AUTO_INCREMENT PRIMARY KEY,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

5.3 orders table

```
CREATE TABLE orders (  
  order_id INT AUTO_INCREMENT PRIMARY KEY,  
  cashier_id INT NOT NULL,  
  total_amount DECIMAL(10, 2) NOT NULL,  
  status ENUM('Pending', 'Reserved', 'Paid', 'Cancelled', 'Expired',  
'Failed') NOT NULL DEFAULT 'Pending',  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
  FOREIGN KEY (cashier_id) REFERENCES cashiers(cashier_id)  
);
```

5.4 order_items table

```
CREATE TABLE order_items (  
  order_item_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL,
```

```

    product_id          INT NOT NULL,
    quantity            INT NOT NULL,
    price_at_purchase   DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

5.5 reservations table

```

CREATE TABLE reservations (
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id       INT NOT NULL,
    product_id     INT NOT NULL,
    quantity       INT NOT NULL,
    reserved_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    expires_at     DATETIME NOT NULL,
    status         ENUM('active', 'expired', 'completed', 'released') NOT
NULL DEFAULT 'active',
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

```

```

-- Note: expires_at = reserved_at + 5 minutes
-- Cron job checks every 60 seconds: if expires_at < NOW() and status =
active -> set expired, restore stock

```

5.6 payments table

```

CREATE TABLE payments (
    payment_id          INT AUTO_INCREMENT PRIMARY KEY,
    order_id            INT NOT NULL UNIQUE,
    method              ENUM('cash', 'card') NOT NULL,
    status              ENUM('success', 'failed', 'timeout') NOT NULL,
    amount              DECIMAL(10, 2) NOT NULL,
    idempotency_key     VARCHAR(100) NOT NULL UNIQUE,
    created_at          DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

```

```

-- idempotency_key prevents duplicate payment submissions for the same order
-- Generated on frontend as UUID when payment form is first opened

```

6. API Endpoints

Base URL: <http://localhost:5000/api>

6.1 Products

Method	Endpoint	Description
GET	/products	Get all products with stock count and price
GET	/products/:id	Get single product details
POST	/products	Add new product (admin only)
PUT	/products/:id	Update product name, price, or stock
DELETE	/products/:id	Delete product (admin only)

6.2 Orders

Method	Endpoint	Description
POST	/orders	Create new order from current item list

GET	/orders/:id	Get order details with items and status
PATCH	/orders/:id/cancel	Cancel an order and restore stock

6.3 Reservations

Method	Endpoint	Description
POST	/reservations	Reserve stock for an order (starts 5-min timer)
DELETE	/reservations/:id	Manually release a reservation and restore stock

6.4 Payments

Method	Endpoint	Description
POST	/payments/process	Mock payment - randomly returns success, failed, or timeout

Request body for POST /payments/process:

```
{
  "order_id": 1,
  "method": "cash" | "card",
  "amount": 1370.00,
  "idempotency_key": "uuid-generated-on-frontend"
}
```

Response outcomes: success -> order status = Paid | failed -> order status = Failed, stock released | timeout -> reservation expired, stock released

7. Key Technical Notes

1. CONCURRENCY: Use MySQL transactions (BEGIN / COMMIT / ROLLBACK) with SELECT ... FOR UPDATE on the products row when checking and deducting stock. This locks the row and prevents two cashiers from purchasing the last item simultaneously.
2. RESERVATION EXPIRY: node-cron runs every minute. Query: UPDATE reservations SET status='expired' WHERE status='active' AND expires_at < NOW(). Then for each expired reservation, restore stock: UPDATE products SET stock_quantity = stock_quantity + reserved_quantity WHERE product_id = ?
3. DUPLICATE PAYMENT: Before processing payment, check if a payment with the same idempotency_key already exists. If yes, return the existing payment result without processing again.
4. ORDER STATUS TRANSITIONS: Pending -> Reserved (on reservation) -> Paid (on success) | Pending -> Reserved -> Failed (on payment failure) | Reserved -> Expired (on timer) | Any active -> Cancelled (on manual cancel)